

多租户核心基础组件实现代码

一、后端可落地实现代码（完全兼容现有Spring Boot + MyBatis-Plus结构）

以下代码均按照你现有「实体-Mapper-Service直接实现-Controller」的分层结构编写，无需调整架构即可接入。

1. 多租户核心基础组件

1.1 租户上下文工具（`util/TenantContext.java`）

基于ThreadLocal存储当前请求的租户ID，请求进出时设置/清除，业务代码无感知。

代码块

```
1  package com.yourproject.util;
2
3  public class TenantContext {
4      private static final ThreadLocal<Long> TENANT_HOLDER = new ThreadLocal<>();
5
6      public static void setTenantId(Long tenantId) {
7          TENANT_HOLDER.set(tenantId);
8      }
9
10     public static Long getTenantId() {
11         return TENANT_HOLDER.get();
12     }
13
14     public static void clear() {
15         TENANT_HOLDER.remove();
16     }
17 }
```

1.2 租户拦截器（`interceptor/TenantInterceptor.java`）

从请求头JWT中解析租户ID，校验租户状态，写入上下文，实现全链路租户透传。

代码块

```
1  package com.yourproject.interceptor;
2
3  import cn.hutool.core.util.StrUtil;
```

```
4 import com.yourproject.entity.Tenant;
5 import com.yourproject.service.TenantService;
6 import com.yourproject.util.JwtUtil;
7 import com.yourproject.util.TenantContext;
8 import org.slf4j.MDC;
9 import org.springframework.beans.factory.annotation.Autowired;
10 import org.springframework.stereotype.Component;
11 import org.springframework.web.servlet.HandlerInterceptor;
12
13 import javax.servlet.http.HttpServletRequest;
14 import javax.servlet.http.HttpServletResponse;
15
16 @Component
17 public class TenantInterceptor implements HandlerInterceptor {
18
19     @Autowired
20     private TenantService tenantService;
21
22     @Override
23     public boolean preHandle(HttpServletRequest request, HttpServletResponse
response, Object handler) throws Exception {
24         String token = request.getHeader("Authorization");
25         if (StrUtil.isBlank(token)) {
26             response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
27             response.setContentType("application/json;charset=UTF-8");
28             response.getWriter().write("{\"code\":401,\"msg\":\"未登录，请先登录
\"}");
29             return false;
30         }
31
32         Long tenantId = JwtUtil.getTenantId(token);
33         Long userId = JwtUtil.getUserId(token);
34         if (tenantId == null || userId == null) {
35             response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
36             response.setContentType("application/json;charset=UTF-8");
37             response.getWriter().write("{\"code\":401,\"msg\":\"登录状态无效，请
重新登录\"}");
38             return false;
39         }
40
41         // 平台管理员 (tenantId=0) 跳租户状态校验
42         if (0L != tenantId) {
43             Tenant tenant = tenantService.getById(tenantId);
44             if (tenant == null || tenant.getStatus() != 1) {
45                 response.setStatus(HttpServletResponse.SC_FORBIDDEN);
46                 response.setContentType("application/json;charset=UTF-8");
```

```

47         response.getWriter().write("{\"code\":403,\"msg\":\"租户已停用或
    不存在,请联系平台管理员\"}");
48         return false;
49     }
50 }
51
52 TenantContext.setTenantId(tenantId);
53 MDC.put("tenantId", String.valueOf(tenantId)); // 日志埋点
54 return true;
55 }
56
57 @Override
58 public void afterCompletion(HttpServletRequest request,
    HttpServletResponse response, Object handler, Exception ex) {
59     TenantContext.clear();
60     MDC.remove("tenantId");
61 }
62 }

```

1.3 MVC拦截器注册 (config/WebMvcConfig.java)

代码块

```

1  package com.yourproject.config;
2
3  import com.yourproject.interceptor.TenantInterceptor;
4  import org.springframework.beans.factory.annotation.Autowired;
5  import org.springframework.context.annotation.Configuration;
6  import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
7  import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
8
9  @Configuration
10 public class WebMvcConfig implements WebMvcConfigurer {
11
12     @Autowired
13     private TenantInterceptor tenantInterceptor;
14
15     @Override
16     public void addInterceptors(InterceptorRegistry registry) {
17         registry.addInterceptor(tenantInterceptor)
18             .addPathPatterns("/api/**")
19             .excludePathPatterns(
20                 "/api/auth/login",
21                 "/api/auth/captcha",
22                 "/static/**",
23                 "/error"

```

```
24         );
25     }
26 }
```

1.4 MyBatis-Plus多租户+分页一体化配置 (config/MyBatisPlusConfig.java)

整合原有分页配置，加入多租户拦截器，自动拼接 `tenant_id` 条件，业务代码零侵入。

代码块

```
1  package com.yourproject.config;
2
3  import com.baomidou.mybatisplus.annotation.DbType;
4  import com.baomidou.mybatisplus.core.conditions.query.LambdaQueryWrapper;
5  import com.baomidou.mybatisplus.extension.plugins.MybatisPlusInterceptor;
6  import
    com.baomidou.mybatisplus.extension.plugins.inner.PaginationInnerInterceptor;
7  import
    com.baomidou.mybatisplus.extension.plugins.inner.TenantLineInnerInterceptor;
8  import com.yourproject.util.TenantContext;
9  import net.sf.jsqlparser.expression.LongValue;
10 import net.sf.jsqlparser.expression.NullValue;
11 import org.springframework.context.annotation.Bean;
12 import org.springframework.context.annotation.Configuration;
13
14 import java.util.Arrays;
15 import java.util.List;
16
17 @Configuration
18 public class MyBatisPlusConfig {
19
20     @Bean
21     public MybatisPlusInterceptor mybatisPlusInterceptor() {
22         MybatisPlusInterceptor interceptor = new MybatisPlusInterceptor();
23
24         // 多租户拦截器（必须放在分页拦截器之前）
25         TenantLineInnerInterceptor tenantInterceptor = new
    TenantLineInnerInterceptor();
26         tenantInterceptor.setTenantLineHandler(new
    com.baomidou.mybatisplus.extension.plugins.handler.TenantLineHandler() {
27             @Override
28             public net.sf.jsqlparser.expression.Expression getTenantId() {
29                 Long tenantId = TenantContext.getTenantId();
30                 // 平台管理员返回null，不拼接租户条件，查询全局数据
31                 return tenantId == null || tenantId == 0 ? new NullValue() :
    new LongValue(tenantId);
32             }
33         });
34     }
35 }
```

```

33
34         @Override
35         public String getTenantIdColumn() {
36             return "tenant_id";
37         }
38
39         @Override
40         public boolean ignoreTable(String tableName) {
41             // 全局平台表不参与租户隔离
42             List<String> ignoreTables = Arrays.asList(
43                 "sys_tenant",
44                 "sys_package",
45                 "sys_platform_admin",
46                 "sys_dict"
47             );
48             return ignoreTables.contains(tableName);
49         }
50     });
51     interceptor.addInnerInterceptor(tenantInterceptor);
52
53     // 分页拦截器
54     interceptor.addInnerInterceptor(new
55     PaginationInnerInterceptor(DbType.MYSQL));
56
57     return interceptor;
58 }

```

2. 租户管理基础代码

2.1 租户实体类（entity/Tenant.java）

对应数据库表 `sys_tenant`

代码块

```

1  package com.yourproject.entity;
2
3  import com.baomidou.mybatisplus.annotation.IdType;
4  import com.baomidou.mybatisplus.annotation.TableId;
5  import com.baomidou.mybatisplus.annotation.TableName;
6  import lombok.Data;
7
8  import java.time.LocalDateTime;
9
10 @Data
11 @TableName("sys_tenant")

```

```

12  public class Tenant {
13      @TableId(type = IdType.AUTO)
14      private Long id;
15      private String tenantCode;
16      private String orgName;
17      private String contact;
18      private String phone;
19      private Integer status;
20      private Long packageId;
21      private LocalDateTime expireTime;
22      private LocalDateTime createTime;
23      private LocalDateTime updateTime;
24  }

```

2.2 租户Mapper (mapper/TenantMapper.java)

代码块

```

1  package com.yourproject.mapper;
2
3  import com.baomidou.mybatisplus.core.mapper.BaseMapper;
4  import com.yourproject.entity.Tenant;
5  import org.apache.ibatis.annotations.Mapper;
6
7  @Mapper
8  public interface TenantMapper extends BaseMapper<Tenant> {
9  }

```

2.3 租户Service (service/TenantService.java)

采用直接实现模式，不定义接口。

代码块

```

1  package com.yourproject.service;
2
3  import com.baomidou.mybatisplus.core.conditions.query.LambdaQueryWrapper;
4  import com.yourproject.entity.Tenant;
5  import com.yourproject.mapper.TenantMapper;
6  import org.springframework.beans.factory.annotation.Autowired;
7  import org.springframework.stereotype.Service;
8
9  @Service
10 public class TenantService {
11
12     @Autowired

```

```

13     private TenantMapper tenantMapper;
14
15     public Tenant getById(Long id) {
16         return tenantMapper.selectById(id);
17     }
18
19     public Tenant getByCode(String tenantCode) {
20         LambdaQueryWrapper<Tenant> wrapper = new LambdaQueryWrapper<>();
21         wrapper.eq(Tenant::getTenantCode, tenantCode);
22         return tenantMapper.selectOne(wrapper);
23     }
24 }

```

3. 认证体系适配

3.1 JWT工具类 (util/JwtUtil.java)

基于Hutool JWT实现，生成携带租户ID、用户ID、角色的令牌。

代码块

```

1  package com.yourproject.util;
2
3  import cn.hutool.jwt.JWT;
4  import cn.hutool.jwt.JWTUtil;
5  import cn.hutool.jwt.signers.JWTSigner;
6  import cn.hutool.jwt.signers.JWTSignerUtil;
7
8  import java.nio.charset.StandardCharsets;
9  import java.util.Date;
10
11 public class JwtUtil {
12     private static final String SECRET = "your-jwt-secret-key-2024";
13     private static final JWTSigner SIGNER =
14         JWTSignerUtil.hs256(SECRET.getBytes(StandardCharsets.UTF_8));
15     private static final long EXPIRE_TIME = 24 * 60 * 60 * 1000L;
16
17     public static String createToken(Long tenantId, Long userId, String role,
18         String userName) {
19         return JWT.create()
20             .setPayload("tenantId", tenantId)
21             .setPayload("userId", userId)
22             .setPayload("role", role)
23             .setPayload("userName", userName)
24             .setExpiresAt(new Date(System.currentTimeMillis() +
25                 EXPIRE_TIME))
26             .sign(SIGNER);
27     }
28 }

```

```

24     }
25
26     public static Long getTenantId(String token) {
27         try {
28             JWT jwt = JWTUtil.parseToken(token);
29             Object val = jwt.getPayload("tenantId");
30             return val == null ? null : Long.valueOf(val.toString());
31         } catch (Exception e) {
32             return null;
33         }
34     }
35
36     public static Long getUserId(String token) {
37         try {
38             JWT jwt = JWTUtil.parseToken(token);
39             Object val = jwt.getPayload("userId");
40             return val == null ? null : Long.valueOf(val.toString());
41         } catch (Exception e) {
42             return null;
43         }
44     }
45
46     public static String getRole(String token) {
47         try {
48             JWT jwt = JWTUtil.parseToken(token);
49             Object val = jwt.getPayload("role");
50             return val == null ? null : val.toString();
51         } catch (Exception e) {
52             return null;
53         }
54     }
55
56     public static boolean verify(String token) {
57         try {
58             return JWTUtil.verify(token, SIGNER);
59         } catch (Exception e) {
60             return false;
61         }
62     }
63 }

```

3.2 登录接口改造 (controller/AuthController.java)

增加租户编码参数，支持多角色统一登录入口。


```
1  package com.yourproject.controller;
2
3  import cn.hutool.core.util.StrUtil;
4  import com.yourproject.entity.Tenant;
5  import com.yourproject.service.TenantService;
6  import com.yourproject.util.JwtUtil;
7  import com.yourproject.util.Result;
8  import org.springframework.beans.factory.annotation.Autowired;
9  import org.springframework.web.bind.annotation.*;
10
11 import java.util.HashMap;
12 import java.util.Map;
13
14 @RestController
15 @RequestMapping("/api/auth")
16 public class AuthController {
17
18     @Autowired
19     private TenantService tenantService;
20
21     @PostMapping("/login")
22     public Result<Map<String, Object>> login(
23         @RequestParam String tenantCode,
24         @RequestParam String username,
25         @RequestParam String password,
26         @RequestParam String role) {
27
28         Long tenantId;
29         String userName;
30
31         // 平台管理员登录
32         if ("platform".equals(tenantCode) && "platform_admin".equals(role)) {
33             if (!"admin".equals(username) || !"admin123".equals(password)) {
34                 return Result.error("账号或密码错误");
35             }
36             tenantId = 0L;
37             userName = "平台管理员";
38         } else {
39             // 租户端登录：先校验租户状态
40             Tenant tenant = tenantService.getByCode(tenantCode);
41             if (tenant == null || tenant.getStatus() != 1) {
42                 return Result.error("租户编码无效或已停用");
43             }
44             tenantId = tenant.getId();
45
46             // 根据角色校验对应账号（教师/学生/租户管理员，补充tenant_id条件查询）
47             if ("teacher".equals(role)) {
```

```

48         // 教师账号校验逻辑
49         userName = "教师用户";
50     } else if ("student".equals(role)) {
51         // 学生账号校验逻辑
52         userName = "学生用户";
53     } else if ("admin".equals(role)) {
54         // 租户管理员校验逻辑
55         userName = "租户管理员";
56     } else {
57         return Result.error("角色类型错误");
58     }
59 }
60
61 String token = JwtUtil.createToken(tenantId, 1L, role, userName);
62
63 Map<String, Object> data = new HashMap<>();
64 data.put("token", token);
65 data.put("role", role);
66 data.put("userName", userName);
67 data.put("tenantCode", tenantCode);
68
69 return Result.success(data);
70 }
71 }

```

4. 业务层改造示例（以CourseService为例）

保留原有直接实现结构，新增套餐权益校验逻辑。

代码块

```

1  @Service
2  public class CourseService {
3
4      @Autowired
5      private CourseMapper courseMapper;
6
7      @Autowired
8      private TenantPackageService tenantPackageService;
9
10     public PageResult<CourseVO> getCoursePage(CourseQuery query) {
11         Page<Course> page = new Page<>(query.getPageNum(),
12         query.getPageSize());
13         LambdaQueryWrapper<Course> wrapper = new LambdaQueryWrapper<>();
14         if (StrUtil.isNotBlank(query.getCourseName())) {
15             wrapper.like(Course::getCourseName, query.getCourseName());

```

```

16         }
17         wrapper.orderByDesc(Course::getCreateTime);
18
19         Page<Course> resultPage = courseMapper.selectPage(page, wrapper);
20         List<CourseVO> voList = BeanUtil.copyToList(resultPage.getRecords(),
CourseVO.class);
21         return PageResult.of(resultPage).setRows(voList);
22     }
23
24     public boolean save(Course course) {
25         // 校验当前租户课程数量是否超出套餐上限
26         int currentCount = courseMapper.selectCount(null);
27         int maxCount =
tenantPackageService.getMaxCourseCount(TenantContext.getTenantId());
28
29         if (currentCount >= maxCount) {
30             throw new BusinessException("课程数量已达套餐上限，请升级套餐后再创建");
31         }
32
33         return courseMapper.insert(course) > 0;
34     }
35 }

```

二、前端可落地实现代码（贴合现有原生JS动态页面结构）

1. 登录页改造（index.html）

在原有表单中增加租户编码输入项。

代码块

```

1 <div class="form-item">
2     <label><i class="fa fa-building"></i> 租户编码</label>
3     <input type="text" id="tenant-code" placeholder="请输入租户编码">
4     <div class="error-tip" id="tenant-error"></div>
5 </div>

```

登录提交逻辑同步修改：

代码块

```

1 async function handleLogin() {
2     const tenantCode = document.getElementById('tenant-code').value.trim();
3     const username = document.getElementById('username').value.trim();

```

```

4      const password = document.getElementById('password').value;
5      const role = document.getElementById('user-role').value;
6
7      if (!tenantCode) {
8          showError('tenant-error', '请输入租户编码');
9          return;
10     }
11
12     const formData = new URLSearchParams();
13     formData.append('tenantCode', tenantCode);
14     formData.append('username', username);
15     formData.append('password', password);
16     formData.append('role', role);
17
18     try {
19         const res = await fetch('/api/auth/login', {
20             method: 'POST',
21             body: formData
22         });
23         const result = await res.json();
24
25         if (result.code === 200) {
26             localStorage.setItem('token', result.data.token);
27             localStorage.setItem('userRole', result.data.role);
28             localStorage.setItem('tenantCode', result.data.tenantCode);
29             localStorage.setItem('userName', result.data.userName);
30
31             redirectByRole(result.data.role);
32         } else {
33             showError('login-error', result.msg);
34         }
35     } catch (e) {
36         showError('login-error', '登录请求失败, 请稍后重试');
37     }
38 }

```

2. 全局请求封装（公共JS文件）

统一封装接口请求，自动携带认证头，统一处理401跳转。

代码块

```

1  const http = {
2      baseUrl: '',
3
4      async get(url, params = {}) {
5          const query = new URLSearchParams(params).toString();

```

```

6      const fullUrl = query ? `${this.baseUrl}${url}?${query}` :
    `${this.baseUrl}${url}`;
7
8      const res = await fetch(fullUrl, {
9          headers: {
10              'Authorization': localStorage.getItem('token') || '',
11              'Content-Type': 'application/json'
12          }
13      });
14      return this.handleResponse(res);
15  },
16
17  async post(url, data = {}) {
18      const res = await fetch(`${this.baseUrl}${url}`, {
19          method: 'POST',
20          headers: {
21              'Authorization': localStorage.getItem('token') || '',
22              'Content-Type': 'application/x-www-form-urlencoded'
23          },
24          body: new URLSearchParams(data)
25      });
26      return this.handleResponse(res);
27  },
28
29  async handleResponse(res) {
30      if (res.status === 401) {
31          localStorage.clear();
32          window.location.href = '/index.html';
33          throw new Error('登录已过期，请重新登录');
34      }
35      const data = await res.json();
36      if (data.code !== 200) {
37          throw new Error(data.msg || '请求失败');
38      }
39      return data;
40  }
41  };

```

使用示例：

代码块

```

1  // 替换原有fetch写法
2  const result = await http.get('/api/course/page', {
3      pageNum: coursePagination.pageNum,
4      pageSize: coursePagination.pageSize,

```

```
5     courseName: document.getElementById('course-name-input').value.trim()
6   });
```

3. 管理端菜单权限适配

根据登录角色动态渲染侧边菜单，区分平台管理员和租户管理员。

代码块

```
1  function loadSideMenu() {
2      const role = localStorage.getItem('userRole');
3      const menuList = document.getElementById('menu-list');
4
5      let menus = [];
6
7      if (role === 'platform_admin') {
8          menus = [
9              { icon: 'fa-tachometer-alt', title: '数据概览', key: 'dashboard' },
10             { icon: 'fa-building', title: '租户管理', key: 'tenant' },
11             { icon: 'fa-th-large', title: '套餐管理', key: 'package' },
12             { icon: 'fa-chart-line', title: '运营统计', key: 'statistics' },
13             { icon: 'fa-cog', title: '系统设置', key: 'settings' }
14         ];
15     } else if (role === 'admin') {
16         menus = [
17             { icon: 'fa-tachometer-alt', title: '首页概览', key: 'dashboard' },
18             { icon: 'fa-book-open', title: '课程管理', key: 'course' },
19             { icon: 'fa-chalkboard-teacher', title: '教师管理', key: 'teacher' },
20             { icon: 'fa-user-graduate', title: '学生管理', key: 'student' },
21             { icon: 'fa-calendar-check', title: '预约管理', key: 'reservation' },
22             { icon: 'fa-chart-bar', title: '数据统计', key: 'report' }
23         ];
24     }
25
26     menuList.innerHTML = menus.map(item => `
27         <li class="menu-item" data-key="${item.key}"
28         onclick="switchMenu('${item.key}')">
29             <i class="fa ${item.icon}"></i>
30             <span>${item.title}</span>
31         </li>
32     `).join('');
```

三、运维监控可落地实现代码

1. 应用监控配置（`application.yml`）

启用Spring Boot Actuator，独立监控端口。

代码块

```
1  management:
2    endpoints:
3      web:
4        exposure:
5          include: health,info,metrics,logfile
6    endpoint:
7      health:
8        show-details: always
9    server:
10     port: 8081
```

可访问监控项：

- 健康检查：<http://localhost:8081/actuator/health>
- 应用指标：<http://localhost:8081/actuator/metrics>
- 实时日志：<http://localhost:8081/actuator/logfile>

2. 带租户标识的日志配置（`logback-spring.xml`）

日志格式内置租户ID，便于按租户排查问题。

代码块

```
1  <configuration>
2    <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
3      <encoder>
4        <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] [%X{tenantId}]
5        %-5level %logger{36} - %msg%n</pattern>
6      </encoder>
7    </appender>
8
9    <root level="info">
10     <appender-ref ref="CONSOLE"/>
11   </root>
12 </configuration>
```

3. 租户级轻量限流（基于Guava）

按租户维度限制请求频率，避免单租户占用全部资源。

代码块

```
1  @Component
2  public class RateLimitInterceptor implements HandlerInterceptor {
3      private final com.google.common.cache.LoadingCache<Long,
4          com.google.common.util.concurrent.RateLimiter> rateLimiterCache =
5          com.google.common.cache.CacheBuilder.newBuilder()
6              .expireAfterAccess(10,
7                  java.util.concurrent.TimeUnit.MINUTES)
8              .build(new com.google.common.cache.CacheLoader<Long,
9                  com.google.common.util.concurrent.RateLimiter>() {
10                  @Override
11                  public com.google.common.util.concurrent.RateLimiter
12                  load(Long tenantId) {
13                      return
14                      com.google.common.util.concurrent.RateLimiter.create(100); // 每秒100请求
15                  }
16              });
17
18      @Override
19      public boolean preHandle(HttpServletRequest request, HttpServletResponse
20          response, Object handler) throws Exception {
21          Long tenantId = TenantContext.getTenantId();
22          if (tenantId == null || tenantId == 0) return true;
23
24          com.google.common.util.concurrent.RateLimiter limiter =
25          rateLimiterCache.get(tenantId);
26          if (!limiter.tryAcquire()) {
27              response.setStatus(429);
28              response.setContentType("application/json;charset=UTF-8");
29              response.getWriter().write("{\"code\":429,\"msg\":\"请求过于频繁，请
30              稍后再试\"}");
31              return false;
32          }
33          return true;
34      }
35  }
```

还需要你提供的数据

为了让代码完全适配你的项目并可直接编译运行，还需要补充以下信息：

项目基础信息

- 项目完整包名（如 `com.language.reservation`）
- Spring Boot、MyBatis-Plus、Hutool的具体版本号
- 现有统一返回结果类 `Result` 的完整包路径和字段结构

数据库相关

- 教师表、学生表、管理员表的完整字段结构
- 所有业务表是否已添加 `tenant_id` 字段及索引
- 平台管理员表、套餐表是否需要从零设计，还是已有初步结构

前端相关

- 现有前端页面的文件目录结构（各HTML、JS文件的存放位置）
- 公共样式、公共JS的文件名和引入方式
- 现有管理端动态内容渲染的入口函数名

业务与部署

- 套餐的具体权益项（课程数量上限、教师数量上限、存储容量等）
- 项目部署方式（jar包运行、Docker、Tomcat）
- 是否需要支持子域名租户识别（如 `xxx.xxx.com` 对应租户）
- 现有短信、文件存储等第三方服务的接入情况

（注：部分内容可能由 AI 生成）